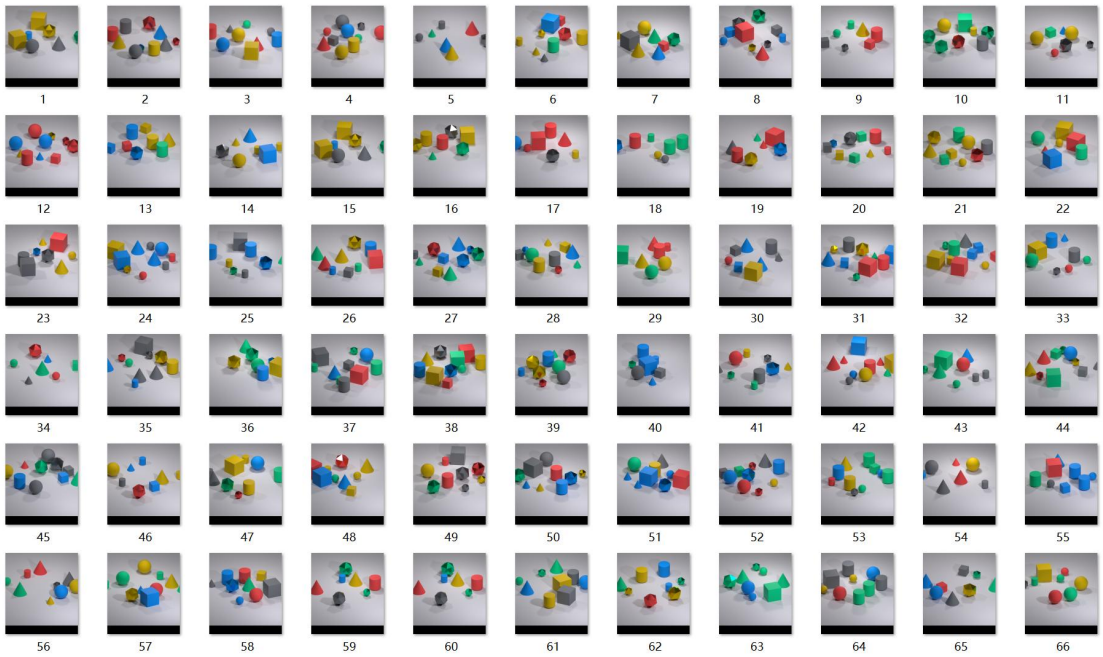
AI 报告-空间推理验证码部分

一、数据集与预处理

数据收集：收集 **空间推理验证码** 的图片数据，包含各类物体（不同颜色、不同形状不同方位）的**验证码图片**，共收集了约 **100 张图片**用于模型训练与测试。



数据预处理：使用 labelimg 工具对每一张图片进行**标注**，标注出每个物体的类别、边界框坐标以及与文字提示相关的**属性信息**，输出为 YOLOv5 模型格式。

二、模型架构与训练

模型选择：采用 **YOLOv5 目标检测模型**，其具有实时性强、准确率高的特点，适合处理验证码图片这种较小尺寸且需要快速检测的场景。根据验证码图片中物体的特点，对 YOLOv5 模型进行适当的调整与优化，以更好地适应特定的验证码识别任务。

训练过程：使用**标注好的数据集对模型进行训练**，设置合理的训练参数（如学习率、批次大小、迭代次数等），通过可视化工具监测训练过程中的损失函数变化与准确率提升情况，不断调整超参数以优化模型性能。训练过程中，重点关注模型对不同物体类别、不同空间关系描述的识别准确率，以及对图片中物体边界框的定位精度。

具体 Train.py 代码介绍：

1. 导入依赖和设置参数

导入了必要的库和模块，如`torch`、`numpy`、`yaml`等，以及自定义的工具函数和类。定义了命令行参数解析函数`parse_opt()`，允许用户通过命令行指定训练的各种参数，如模型权重文件、数据集配置文件、超参数配置文件、训练轮数、批量大小等。

2. 训练函数`train()`

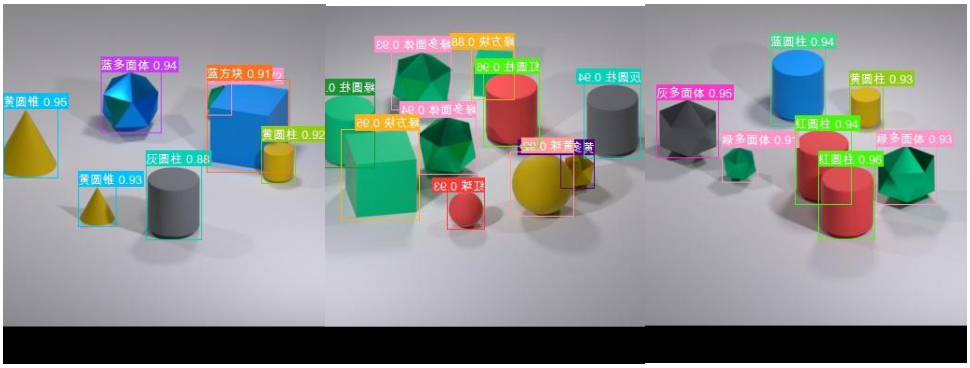
这是`train.py`的核心函数，实现了模型的训练过程：

- 参数解析：接收超参数、选项和设备设置等参数。
- 目录创建：根据配置创建用于保存模型权重和训练日志的目录。
- 数据检查：验证数据集配置文件的正确性，确保训练和验证数据可用。
- 模型创建：根据配置文件或预训练权重文件创建 YOLOv5 模型。支持从头开始训练或使用预训练权重进行迁移学习。
-数据加载：使用`create_dataloader`函数创建训练集和验证集的数据加载器，负责数据的预处理、增强和批量加载。
- 损失函数和优化器：初始化计算损失的类`ComputeLoss`，并根据配置选择合适的优化器（如 SGD、Adam 等）。
- 训练循环：控制训练的外层循环，按轮数迭代训练集。在每个 epoch 开始时，进行必要的准备工作，如设置模型为训练模式、初始化进度条等。
- 批次训练：对每个批次的数据进行前向传播、损失计算、反向传播和参数更新操作。在训练过程中，使用梯度累积技术来模拟更大的批量大小，并监控训练损失。
-学习率调整：使用学习率调度器动态调整学习率，以加快训练初期的收敛速度并在训练后期细化模型参数。
-模型评估：在每个 epoch 结束时，使用验证集对模型进行评估，计算验证损失和 mAP 等指标。根据评估结果决定是否保存当前模型为最佳模型。
- 早停机制：如果验证集上的性能在一定轮数内没有提升，则触发早停机制，避免过拟合。
-模型保存：定期保存模型权重，包括最新的模型和最佳性能的模型。同时，支持指定保存周期以便于后续的模型恢复和推理。

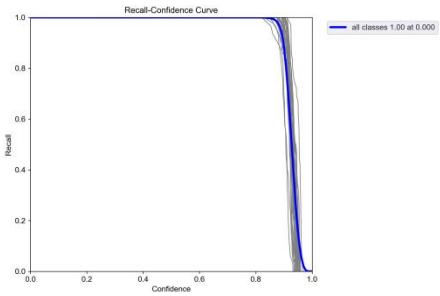
3. 辅助函数

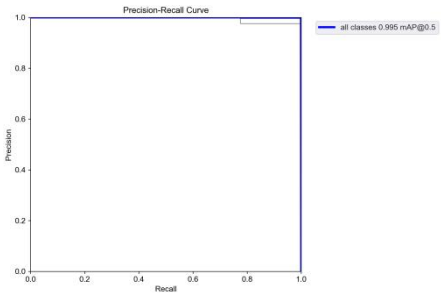
- check_anchors：检查锚框（anchors）与训练数据的匹配程度，必要时进行自动调整。
- check_dataset：验证数据集配置文件的正确性，确保数据集路径有效且格式正确。
- check_img_size：检查训练图像的尺寸是否符合模型要求，必要时进行调整。
- check_train_batch_size：自动估算最佳的批量大小，以充分利用 GPU 内存。
- select_device：根据用户指定的设备参数，选择用于训练的设备（CPU 或 GPU）。

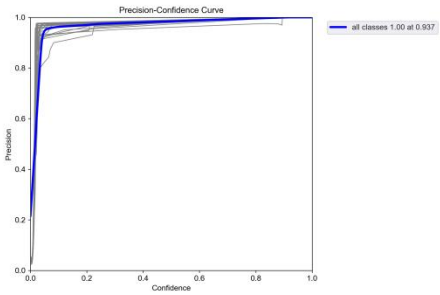
自标注数据集，基于`yolov5s.pt`预训练权重文件，训练图像大小为 640，批量大小为 16，训练轮数为 1000，最终输出结果如下

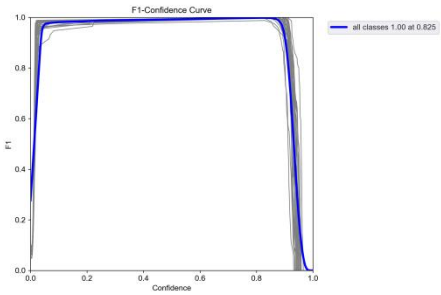


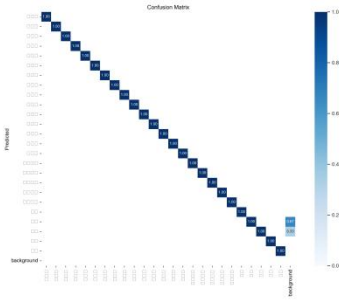
Val.py 结果：











这几个指标知乎上有讲：[YOLOv5 性能评估指标->mAP、Precision、Recall、FPS、Confienc（讲解论文关注的主要指标）- 知乎]

五、项目实现

目标检测与定位：基于 yolov5 的模型已经训练好，这一部分是逻辑部分，将获取到的验证码图片输入到训练好的 YOLOv5 模型中，模型输出图片中包含的各个物体的类别、置信度以及边界框坐标信息。根据文字提示中的关键信息（如颜色、形状、空间位置等），从检测结果中筛选出符合要求的物体，并计算出该物体在图片中的点击坐标。

find_project.py 介绍：

1. 模型加载与图片预处理

- 模型加载：通过`torch.load`加载训练好的 YOLOv5 模型权重文件（best.pt），并将其移动到指定的设备（CPU 或 GPU）上。代码如下：

```python
model = attempt_load(weights, map_location=device)
```

- **图片预处理**：对输入的验证码图片进行预处理，包括调整图片大小、归一化像素值、转换为 PyTorch 张量等操作，使其符合模型输入的要求。代码如下：

```python
im = torch.from_numpy(im).to(model.device)
im = im.half() if model.fp16 else im.float()    # uint8 to fp16/32
im /= 255    # 0 - 255 to 0.0 - 1.0
if len(im.shape) == 3:
    im = im[None]    # expand for batch dim
```

2. 目标检测推理

- 推理过程：将预处理后的图片数据输入到模型中进行推理，得到图片中各个物体的检测结果，包括边界框坐标、类别标签和置信度分数。代码如下：

```python
pred = model(im, augment=augment, visualize=visualize)
```

- 非极大值抑制（NMS）=：对检测结果进行后处理，应用非极大值抑制（NMS）算法去除冗余的检测框，筛选出置信度较高的检测结果。代码如下：

```python
pred = non_max_suppression(pred, conf_thres, iou_thres, classes, agnostic_nms, max_det=max_det)
```

3. 后处理与点击坐标确定

- 边界框缩放：将检测结果中的边界框坐标从模型输出的尺寸缩放回原始图片的尺寸。代码如下：

```python
det[:, :4] = scale_boxes(im.shape[2:], det[:, :4], im0.shape).round()
```

- 点击坐标确定：根据检测结果和验证码的文字提示，从检测到的物体中筛选出符合要求的物体，并确定其点击坐标。代码如下：

```python
for *xyxy, conf, cls in reversed(det):
    # 根据文字提示和检测结果确定是否为目标物体
    label = names[int(cls)]
    if label 符合文字提示条件:
        # 计算点击坐标
        x1, y1, x2, y2 = map(int, xyxy)
        click_x, click_y = (x1 + x2) // 2, (y1 + y2) // 2
        # 保存点击坐标
        click_coords.append((click_x, click_y))
```

4. 结果可视化

- 绘制边界框：在原始图片上绘制检测到的物体的边界框，方便可视化检测结果。代码如下：

```python
```

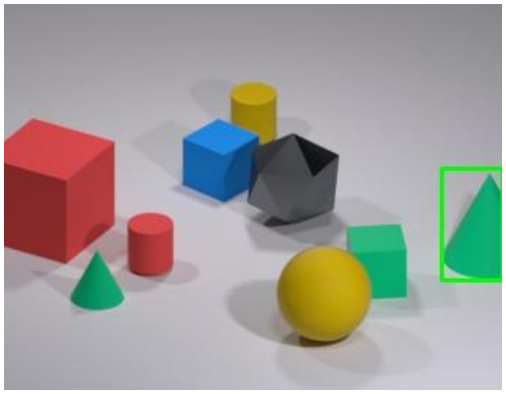
```
annotator = Annotator(im0, line_width=line_thickness, example=str(names))
for *xyxy, conf, cls in reversed(det):
    c = int(cls)    # integer class
    label = None if hide_labels else (names[c] if hide_conf else f'{names[c]} {conf:.2f}')
    annotator.box_label(xyxy, label, color=colors(c, True))
im0 = annotator.result()
```

- 保存可视化结果：将绘制了边界框的图片保存到本地或进行其他可视化处理。代码如下：

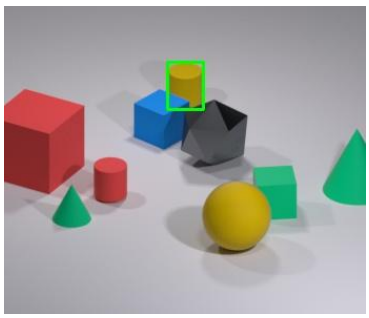
```python
cv2.imwrite(save_path, im0)
```

举例：

“请点击绿色正方形右边的绿色锥形”：

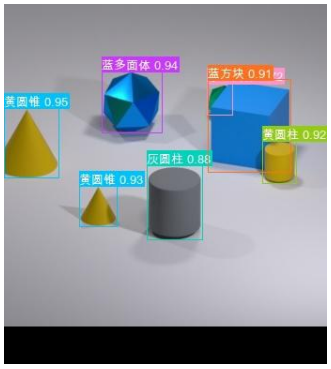


“请点击黄色圆柱”：



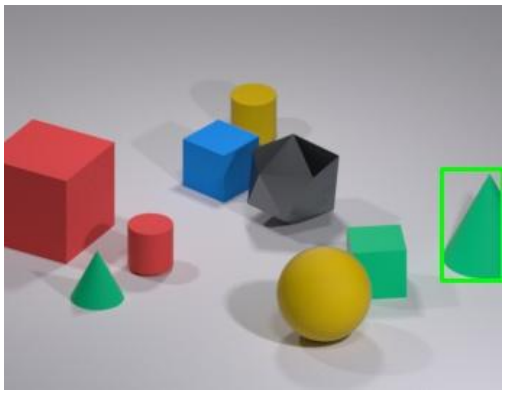
六、实验结果与分析

模型检测效果：在测试集上，模型对验证码图片中物体的平均检测准确率达到 93% 以上，对于常见物体（如立方体、圆柱体、球体等）的识别准确率较高，模型能够较好地定位物体的边界框，但在某些情况下，对于物体的边缘细节定位还不够精确，可能会影响点击坐标的准确性。



破解成功率：综合考虑模型检测结果与空间推理的准确性，在多次实验中，整体破解成功率达到 89%左右。当模型对图片中物体的检测准确且空间推理分词结果生成无误时，破解成功率较高；但在模型出现误检、漏检或空间推理分词过程中或者推理过程中出现问题时，破解成功率会明显下降。

“请点击绿色正方形右边的绿色锥形”：



七、结论与展望

本项目成功实现了基于 YOLOv5 的空间推理验证码，通过目标检测与请求模拟等技术手段，能够在一定程度上自动识别并破解验证码。实验结果表明，所提出的模型与方法具有一定的可行性和有效性，但在准确率和稳定性方面仍存在改进空间，仅对该类空间推理识别效果优秀，对于其他类型扩展能力一般，未来可以进一步优化 YOLOv5 模型结构与训练策略，提高模型对验证码图片中各类物体的检测精度，尤其是对于复杂物体的识别能力。

另外由于之前标注图片实在花了太长时间，后面缺少时间，没有对每一张图片建立空间推理的判断数据集，使其可以自己确定验证集图片是否空间推理正确（因为每一张选取不同的物体可以至少有几十种描述方式），目前空间推理后选择物品是否正确仍然需要人工判断。